

Lab 1: Type Definition and Function Encapsulation

```
CREATE DATABASE Advance_Database;
```

– Case 1:

```
CREATE TABLE Advance_Database..Student(  
roll_number int,  
First_Name varchar(30),  
Mid_Name Varchar(30),  
Last_Name varchar(30),  
Contact_No varchar(10),  
email varchar(40),  
DOB date  
);
```

```
SELECT * FROM Advance_Database..Student;
```

Results Messages							
roll_number	First_Name	Mid_Name	Last_Name	Contact_No	email	DOB	

```
Insert into Advance_Database..Student values;  
(1,'Ram','','Singh',123456,'', '2002-05-12');
```

```
SELECT * FROM Advance_Database..Student;
```

roll_number	First_Name	Mid_Name	Last_Name	Contact_No	email	DOB	
1	Ram		Singh	123456		2002-05-12	

– Case 2

```
CREATE TABLE Student2(  
roll INT,  
First_Name VARCHAR(30),  
Mid_Name VARCHAR(30),  
Last_Name VARCHAR(30),  
contact_no VARCHAR(10),  
email VARCHAR(30),  
DOB DATE  
);
```

Results	Messages						
	roll	First_Name	Mid_Name	Last_Name	contact_no	email	DOB

```
INSERT INTO Student2
VALUES (1,'Ram','', 'Singh','123456','',GETDATE());
```

```
SELECT * FROM student2;
```

Results	Messages						
	roll	First_Name	Mid_Name	Last_Name	contact_no	email	DOB
1	1	Ram		Singh	123456		2026-03-10

– Case 3:

```
CREATE FUNCTION getAge (@dob DATE)
RETURNS INT
AS
BEGIN
    RETURN DATEDIFF(YEAR, @dob, GETDATE());
END;
```

```
CREATE TABLE Student3(
    roll INT,
    First_Name VARCHAR(30),
    Mid_Name VARCHAR(30),
    Last_Name VARCHAR(30),
    DOB DATE,
    contact_no VARCHAR(10),
    email VARCHAR(100)
);
```

```
INSERT INTO Student3
VALUES (1,'Ram','Kumar','Singh','1980-01-01','123456','ram@testmail.com');
```

SELECT * FROM Student3;

Results		Messages						
	roll	First_Name	Mid_Name	Last_Name	DOB	contact_no	email	
1	1	Ram	Kumar	Singh	1980-01-01	123456	ram@testmail.com	

– Select with Age Calculation

```
SELECT
roll,
First_Name,
Mid_Name,
Last_Name,
DOB,
FORMAT(DOB,'dd/MMM/yyyy') AS Formatted_DOB,
contact_no,
email,
dbo.getAge(DOB) AS Age
FROM Student3;
```

Results		Messages							
	roll	First_Name	Mid_Name	Last_Name	DOB	Formatted_DOB	contact_no	email	Age
1	1	Ram	Kumar	Singh	1980-01-01	01/Jan/1980	123456	ram@testmail.com	46

– Final Output Query

```
SELECT
roll,
First_Name,
Mid_Name,
Last_Name,
DOB,
contact_no,
email,
dbo.getAge(DOB) AS Age
FROM Student3;
```

Results		Messages							
	roll	First_Name	Mid_Name	Last_Name	DOB	contact_no	email	Age	
1	1	Ram	Kumar	Singh	1980-01-01	123456	ram@testmail.com	46	

Lab 2: Class Hierarchy Atom list tuple

-- Case 1- Create Address Table

```
CREATE TABLE Address2 (  
    address_id INT IDENTITY(1,1) PRIMARY KEY,  
    street VARCHAR(60),  
    city VARCHAR(30),  
    state CHAR(2),  
    zip_code CHAR(5)  
);
```

– Insert Address

```
INSERT INTO Address2 (street, city, state, zip_code)  
VALUES ('Sanepa', 'Lalitpur', 'BG', 'LAL');
```

```
SELECT * FROM Address2;
```

	address_id	street	city	state	zip_code
	1	Sanepa	Lalitpur	BG	LAL

– Case 2: Create Person Table

```
CREATE TABLE Person2 (  
    person_id INT IDENTITY(1,1) PRIMARY KEY,  
    name VARCHAR(30),  
    ssn INT,  
    address_id INT,  
    FOREIGN KEY (address_id) REFERENCES Address2(address_id)  
);
```

– Insert Person

```
INSERT INTO Person2 (name, ssn, address_id)  
VALUES ('Ram', 123456, 1);
```

```
SELECT * FROM Person2;
```

	person_id	name	ssn	address_id
1	1	Ram	123456	1

```
-- Employee Table
CREATE TABLE Employees2 (
    empnumber INT PRIMARY KEY,
    person_id INT,
    manager_id INT,
    office_address_id INT,
    salary DECIMAL(10,2),
    FOREIGN KEY (person_id) REFERENCES Person2(person_id),
    FOREIGN KEY (manager_id) REFERENCES Person2(person_id),
    FOREIGN KEY (office_address_id) REFERENCES Address2(address_id)
);
```

```
-- Insert Employee
INSERT INTO Employees2 (empnumber, person_id, salary)
VALUES (1,1,50000);
```

```
SELECT * FROM Employees2;
```

	empnumber	person_id	manager_id	office_address_id	salary
1	1	1	NULL	NULL	50000.00

```
-- Phone Numbers Table
CREATE TABLE Phone_Array (
    phone_id INT IDENTITY(1,1) PRIMARY KEY,
    empnumber INT,
    phone_number VARCHAR(30),
    FOREIGN KEY (empnumber) REFERENCES Employees2(empnumber)
);
```

```
-- Insert Phone Numbers
INSERT INTO Phone_Array (empnumber, phone_number)
VALUES
(1,'1'),(1,'2'),(1,'3'),(1,'4'),(1,'5');
```

```
SELECT * FROM Phone_Array;
```

	phone_id	empnumber	phone_number
1	1	1	1
2	2	1	2
3	3	1	3
4	4	1	4
5	5	1	5

– Multiple Address Table

```
CREATE TABLE Employee_Address (  
  id INT IDENTITY(1,1) PRIMARY KEY,  
  empnumber INT,  
  address_id INT,  
  FOREIGN KEY (empnumber) REFERENCES Employees2(empnumber),  
  FOREIGN KEY (address_id) REFERENCES Address2(address_id)  
);
```

-- Insert Multiple Addresses

```
INSERT INTO Employee_Address (empnumber, address_id)  
VALUES (1,1);
```

```
SELECT * FROM Employee_Address;
```

	id	empnumber	address_id
1	1	1	1

Lab 4: JSON Example

-- Create DEPT Table

```
CREATE TABLE DEPT (  
  DEPTNO INT PRIMARY KEY,  
  DNAME VARCHAR(14),  
  LOC VARCHAR(13)  
);
```

-- Insert Department Data

```
INSERT INTO DEPT VALUES (10,'ACCOUNTING','NEW YORK');  
INSERT INTO DEPT VALUES (20,'RESEARCH','DALLAS');  
INSERT INTO DEPT VALUES (30,'SALES','CHICAGO');  
INSERT INTO DEPT VALUES (40,'OPERATIONS','BOSTON');
```

SELECT * FROM DEPT;

	DEPTNO	DNAME	LOC
1	10	ACCOUNTING	NEW YORK
2	20	RESEARCH	DALLAS
3	30	SALES	CHICAGO
4	40	OPERATIONS	BOSTON

-- Create EMP Table

```
CREATE TABLE EMP (  
  EMPNO INT PRIMARY KEY,  
  ENAME VARCHAR(10),  
  JOB VARCHAR(9),  
  MGR INT,  
  HIREDATE DATE,  
  SAL DECIMAL(7,2),  
  COMM DECIMAL(7,2),  
  DEPTNO INT,  
  CONSTRAINT FK_DEPTNO FOREIGN KEY (DEPTNO)  
  REFERENCES DEPT(DEPTNO)  
);
```

-- Insert Employee Data

```

INSERT INTO EMP VALUES (7369,'SMITH','CLERK',7902,'1980-12-17',800,NULL,20);
INSERT INTO EMP VALUES (7499,'ALLEN','SALESMAN',7698,'1981-02-20',1600,300,30);
INSERT INTO EMP VALUES (7521,'WARD','SALESMAN',7698,'1981-02-22',1250,500,30);
INSERT INTO EMP VALUES (7566,'JONES','MANAGER',7839,'1981-04-02',2975,NULL,20);
INSERT INTO EMP VALUES (7654,'MARTIN','SALESMAN',7698,'1981-09-28',1250,1400,30);
INSERT INTO EMP VALUES (7698,'BLAKE','MANAGER',7839,'1981-05-01',2850,NULL,30);
INSERT INTO EMP VALUES (7782,'CLARK','MANAGER',7839,'1981-06-09',2450,NULL,10);
INSERT INTO EMP VALUES (7788,'SCOTT','ANALYST',7566,'1987-07-13',3000,NULL,20);
INSERT INTO EMP VALUES (7839,'KING','PRESIDENT',NULL,'1981-11-17',5000,NULL,10);
INSERT INTO EMP VALUES (7844,'TURNER','SALESMAN',7698,'1981-09-08',1500,0,30);
INSERT INTO EMP VALUES (7876,'ADAMS','CLERK',7788,'1987-07-13',1100,NULL,20);
INSERT INTO EMP VALUES (7900,'JAMES','CLERK',7698,'1981-12-03',950,NULL,30);
INSERT INTO EMP VALUES (7902,'FORD','ANALYST',7566,'1981-12-03',3000,NULL,20);
INSERT INTO EMP VALUES (7934,'MILLER','CLERK',7782,'1982-01-23',1300,NULL,10);

```

SELECT * FROM EMP;

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
1	7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
2	7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30
3	7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30
4	7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
5	7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30
6	7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7	7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10
8	7788	SCOTT	ANALYST	7566	1987-07-13	3000.00	NULL	20
9	7839	KING	PRESIDE...	NU...	1981-11-17	5000.00	NULL	10
10	7844	TURN...	SALESMAN	7698	1981-09-08	1500.00	0.00	30
11	7876	ADAMS	CLERK	7788	1987-07-13	1100.00	NULL	20
12	7900	JAMES	CLERK	7698	1981-12-03	950.00	NULL	30
13	7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20
14	7934	MILLER	CLERK	7782	1982-01-23	1300.00	NULL	10

-- Create JSON Table

```
CREATE TABLE json_documents (  
  id INT,  
  data NVARCHAR(MAX)  
);
```

-- Insert JSON Data from EMP Table

```
INSERT INTO json_documents (id, data)  
SELECT  
ROW_NUMBER() OVER(ORDER BY empno),  
(  
  SELECT empno, ename, job  
  FOR JSON PATH, WITHOUT_ARRAY_WRAPPER  
)  
FROM EMP;
```

-- View JSON Data

```
SELECT * FROM json_documents;
```

	id	data
1	1	{"empno":7369,"ename":"SMITH","job":"CLERK"}
2	2	{"empno":7499,"ename":"ALLEN","job":"SALESMAN"}
3	3	{"empno":7521,"ename":"WARD","job":"SALESMAN"}
4	4	{"empno":7566,"ename":"JONES","job":"MANAGER"}
5	5	{"empno":7654,"ename":"MARTIN","job":"SALESMAN"}
6	6	{"empno":7698,"ename":"BLAKE","job":"MANAGER"}
7	7	{"empno":7782,"ename":"CLARK","job":"MANAGER"}
8	8	{"empno":7788,"ename":"SCOTT","job":"ANALYST"}
9	9	{"empno":7839,"ename":"KING","job":"PRESIDENT"}
10	10	{"empno":7844,"ename":"TURNER","job":"SALESMAN"}
11	11	{"empno":7876,"ename":"ADAMS","job":"CLERK"}
12	12	{"empno":7900,"ename":"JAMES","job":"CLERK"}
13	13	{"empno":7902,"ename":"FORD","job":"ANALYST"}
14	14	{"empno":7934,"ename":"MILLER","job":"CLERK"}

-- Extract JSON Values

```
SELECT
JSON_VALUE(data,'$.empno') AS EMPNO,
JSON_VALUE(data,'$.ename') AS ENAME,
JSON_VALUE(data,'$.job') AS JOB
FROM json_documents;
```

	EMPNO	ENAME	JOB
1	7369	SMITH	CLERK
2	7499	ALLEN	SALESMAN
3	7521	WARD	SALESMAN
4	7566	JONES	MANAGER
5	7654	MARTIN	SALESMAN
6	7698	BLAKE	MANAGER
7	7782	CLARK	MANAGER
8	7788	SCOTT	ANALYST
9	7839	KING	PRESIDENT
10	7844	TURNER	SALESMAN
11	7876	ADAMS	CLERK
12	7900	JAMES	CLERK
13	7902	FORD	ANALYST
14	7934	MILLER	CLERK

-- JSON Array Table

```
CREATE TABLE json_documents_array (
  id INT,
  data NVARCHAR(MAX)
);
```

-- Insert JSON Array

```
INSERT INTO json_documents_array (id,data)
VALUES
(1,
(
SELECT empno,ename,job
FROM EMP
FOR JSON PATH
)));
```

-- View JSON Array

```
SELECT * FROM json_documents_array;
```

	id	data
1	1	[{"empno":7369,"ename":"SMITH","job":"CLERK"},{"e...

Lab 5: Trigger Implementation for ECA Model

-- Create Transaction Table

```
CREATE TABLE employee(  
empid INT,  
empname VARCHAR(50),  
address VARCHAR(100),  
contactno VARCHAR(10)  
);
```

EXEC sp_help employee;

	Name	Owner	Type	Created_datetime						
1	employee	dbo	user table	2026-03-10 21:05:52.783						

	Column_name	Type	Computed	Length	Prec	Scale	Nullable	TrimTrailingBlanks	FixedLenNullInSource	Collation
1	empid	int	no	4	10	0	yes	(n/a)	(n/a)	NULL
2	empname	varchar	no	50			yes	no	yes	SQL_Latin1_General_CP1_CI_AS
3	address	varchar	no	100			yes	no	yes	SQL_Latin1_General_CP1_CI_AS
4	contactno	varchar	no	10			yes	no	yes	SQL_Latin1_General_CP1_CI_AS

Identity		Seed	Increment	Not For Replication
1	No identity column defined.	NULL	NULL	NULL

RowGuidCol	
1	No rowguidcol column defined.

Data_located_on_filegroup	
1	PRIMARY

-- Create Audit Table

```
CREATE TABLE audit_trail(  
empid_val INT,  
empname_val VARCHAR(50),  
op_date DATETIME,  
operation VARCHAR(100)  
);
```

-- Insert Data

```
INSERT INTO employee  
VALUES (1,'Ram Singh','KTM','12345');
```

-- View Table

```
SELECT * FROM employee;
```

	empid	empname	address	contactno
1	1	Ram Singh	KTM	12345

```
SELECT * FROM audit_trail;
```

```
-- Create Trigger (Post Insert Event)
```

```
CREATE TRIGGER trig_audittrail
```

```
ON employee
```

```
AFTER INSERT
```

```
AS
```

```
BEGIN
```

```
INSERT INTO audit_trail(
```

```
empid_val,
```

```
empname_val,
```

```
op_date,
```

```
operation
```

```
)
```

```
SELECT
```

```
empid,
```

```
empname,
```

```
GETDATE(),
```

```
'Record Insertion'
```

```
FROM INSERTED;
```

```
END;
```

```
-- Test Trigger
```

```
INSERT INTO employee
```

```
VALUES (2,'Sita','BKT','7678687');
```

```
-- Check Audit Trail
```

```
SELECT * FROM employee;
```

```
SELECT * FROM audit_trail;
```

	empid_val	empname_val	op_date	operation
1	2	Sita	2026-03-10 21:12:58.177	Record Insertion

Lab 6: Temporal Database

-- Create Temporal Table

```
CREATE TABLE Student5(  
rollno INT PRIMARY KEY,  
name VARCHAR(50),  
address VARCHAR(50),  
contactno VARCHAR(10),
```

```
SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START,
```

```
SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END,
```

```
PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime)
```

```
)
```

```
WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = dbo.Student5History));
```

-- Insert Record

```
INSERT INTO Student5 (rollno,name,address,contactno)
```

```
VALUES (1,'Hari','KTM','1234');
```

```
SELECT * FROM Student5;
```

	rollno	name	address	contactno	SysStartTime	SysEndTime
1	1	Hari	KTM	1234	2026-03-10 15:44:48.4564596	9999-12-31 23:59:59.9999999

```
UPDATE Student5
```

```
SET name = 'Ram'
```

```
WHERE rollno = 1;
```

```
UPDATE Student5
```

```
SET name = 'Mohan'
```

```
WHERE rollno = 1;
```

```
UPDATE Student5
```

```
SET name = 'Geeta'
```

```
WHERE rollno = 1;
```

```
SELECT * FROM Student5;
```

	rollno	name	address	contactno	SysStartTime	SysEndTime
1	1	Geeta	KTM	1234	2026-03-10 15:46:27.2413653	9999-12-31 23:59:59.9999999

-- View Version History (Temporal Query)

```
SELECT *  
FROM Student5  
FOR SYSTEM_TIME ALL  
WHERE rollno = 1;
```

	rollno	name	address	contactno	SysStartTime	SysEndTime
1	1	Geeta	KTM	1234	2026-03-10 15:46:27.2413653	9999-12-31 23:59:59.9999999
2	1	Hari	KTM	1234	2026-03-10 15:44:48.4564596	2026-03-10 15:46:27.2373116
3	1	Ram	KTM	1234	2026-03-10 15:46:27.2373116	2026-03-10 15:46:27.2393408
4	1	Mohan	KTM	1234	2026-03-10 15:46:27.2393408	2026-03-10 15:46:27.2413653

-- Show Start and End Time

```
SELECT  
rollno,  
name,  
SysStartTime AS START_DATE,  
SysEndTime AS END_DATE  
FROM Student5  
FOR SYSTEM_TIME ALL  
WHERE rollno = 1;
```

	rollno	name	START_DATE	END_DATE
1	1	Geeta	2026-03-10 15:46:27.2413653	9999-12-31 23:59:59.9999999
2	1	Hari	2026-03-10 15:44:48.4564596	2026-03-10 15:46:27.2373116
3	1	Ram	2026-03-10 15:46:27.2373116	2026-03-10 15:46:27.2393408
4	1	Mohan	2026-03-10 15:46:27.2393408	2026-03-10 15:46:27.2413653

-- View History table

```
SELECT * FROM Student5History;
```

	rollno	name	address	contactno	SysStartTime	SysEndTime
1	1	Hari	KTM	1234	2026-03-10 15:44:48.4564596	2026-03-10 15:46:27.2373116
2	1	Ram	KTM	1234	2026-03-10 15:46:27.2373116	2026-03-10 15:46:27.2393408
3	1	Mohan	KTM	1234	2026-03-10 15:46:27.2393408	2026-03-10 15:46:27.2413653

Lab 7: Function Package: Concept of Function, Procedure, Package and their use

```
-- Case 1: Fetch Person Name
CREATE FUNCTION dbo.Fetch_Person_Name (@Emp_ID INT)
RETURNS VARCHAR(120)
AS
BEGIN
    DECLARE @Name VARCHAR(120)

    SELECT @Name =
        ISNULL(First_Name, '') + ' ' +
        ISNULL(Mid_Name, '') + ' ' +
        ISNULL>Last_Name, '')
    FROM dbo.Personnel_Detail
    WHERE Emp_Number = @Emp_ID

    RETURN @Name
END
GO
```

```
-- Case 2: Fetch Appointed Unit Code
CREATE FUNCTION dbo.Fetch_Appoint_Unit (@Emp_ID INT)
RETURNS VARCHAR(3)
AS
BEGIN
    DECLARE @Unit VARCHAR(3)

    SELECT @Unit = Appoint_Unit
    FROM dbo.Personnel_Detail
    WHERE Emp_Number = @Emp_ID

    RETURN @Unit
END
GO
```

```

-- Case 3: Fetch Appointed Unit Name
CREATE FUNCTION dbo.Appoint_Unit_Name (@Emp_ID INT)
RETURNS VARCHAR(80)
AS
BEGIN
    DECLARE @Unit_Name VARCHAR(80)

    SELECT @Unit_Name = U.Unit_Name
    FROM dbo.Personnel_Detail P
    JOIN dbo.Unit_Master U
        ON P.Appoint_Unit = U.Unit_Code
    WHERE P.Emp_Number = @Emp_ID

    RETURN @Unit_Name
END
GO

```

```

-- Case 4: Fetch Appointed Post Code
CREATE FUNCTION dbo.Appoint_Post_Code (@Emp_ID INT)
RETURNS VARCHAR(3)
AS
BEGIN
    DECLARE @Post VARCHAR(3)

    SELECT @Post = Appoint_Post
    FROM dbo.Personnel_Detail
    WHERE Emp_Number = @Emp_ID

    RETURN @Post
END
GO

```

```

-- Example: Get Name for One Employee
SELECT dbo.Fetch_Person_Name(101) AS Employee_Name;

```

```

-- Example 2: Retrieve Full Employee Information
SELECT
    Emp_Number,
    dbo.Fetch_Person_Name(Emp_Number) AS Employee_Name,
    dbo.Fetch_Appoint_Unit(Emp_Number) AS Unit_Code,
    dbo.Appoint_Unit_Name(Emp_Number) AS Unit_Name,
    dbo.Appoint_Post_Code(Emp_Number) AS Post_Code

```


FROM dbo.Personnel_Detail;

Emp_Number	Employee_Name	Unit_Code	Unit_Name	Post_Code
101	Ram Kumar Sharma	HR	Human Resource	M01
102	Sita Devi Karki	FIN	Finance	A02

TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



Lab Report of
Advance Database

A Lab Report Submitted as required for Advance Database

Bachelor of Science in Computer Science & Information Technology (B.SC. CSIT)

8th Semester of Tribhuvan University, Nepal

Submitted By:
Aayush Basnet
29131/078

Submitted to:
Raheem Ansari

March 12, 2026

SN	Name of Practical	Date of Practical	Signature
1.			
2.			
3.			
4.			
5.			
6.			

Lab 3: Mongo DB Implementation

1. Create new database and collection

```
current MongoDB log ID: 7220f5adb06122fbc9d1d2
connecting to: mongod://127.0.0.1:27017/?directconnection=true&serverSelectionTimeoutMS=2000&appName=mongo-v2.7.0
using MongoDB: 4.0.19
using MongoSh: 2.7.0

test> show dbs
admin    40.00 Kib
config  40.00 Kib
local   72.00 Kib

test> use sbmcst
switched to db sbmcst

sbmcst> db.createCollection("subject")
{ ok: 1 }

sbmcst> show collections
subject

sbmcst> db.subject.drop()
true

sbmcst> db.createCollection("comments")
{ ok: 1 }
```

2. Inserting documents on comment collection

```
sbmcst> db.createCollection('comments')
{ ok: 1 }

sbmcst> db.comments.insertOne({'name': 'nury', 'lang': 'javascript', 'member_since': 5})
{
  acknowledged: true,
  insertedId: ObjectId("55e0b3d0f1d1b1a4ce0f70")
}

sbmcst> db.comments.insertMany([{'name': 'Nohan', 'lang': 'Python', 'member_since': 3}, {'name': 'Ivish', 'lang': 'Java', 'member_since': 4}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId("55e0b3d0f1d1b1a4ce0f72"),
    '1': ObjectId("55e0b3d0f1d1b1a4ce0f73")
  }
}
```

3. Displaying documents on comment collection

```
mbmcst> db.comments.find().pretty()
[
  {
    _id: ObjectId("65ea9b1c8f1d2b3a4c5e6f70"),
    name: 'Harry',
    lang: 'JavaScript',
    member_since: 5
  },
  {
    _id: ObjectId("65ea9b2d8f1d2b3a4c5e6f72"),
    name: 'Rohan',
    lang: 'Python',
    member_since: 3
  },
  {
    _id: ObjectId("65ea9b2d8f1d2b3a4c5e6f73"),
    name: 'Lovish',
    lang: 'Java',
    member_since: 4
  }
]
```

4. Updating document

```
mbmcst> db.comments.updateOne({name: 'Rohan'}, {$inc: {member_since: 2}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

mbmcst> db.comments.updateOne({name: 'Rohan'}, {$rename: {member_since: 'member'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

5. Deleting document from comments collection

```
mbmcst> db.comments.deleteOne({name: 'Harry'})
{ acknowledged: true, deletedCount: 1 }
```